

MY STUDENT ACADEMIA

Course Registration & Academic Management System

Comprehensive Software Engineering Design Specification & Technical Architecture

Domain / Category:	Academic Course Management & Student Information Systems (SIS)
Software Process Model:	Agile Scrum with Incremental Iterative Delivery
Architecture Style:	3-Tier Layered & Client-Server RESTful Architecture
Primary Stakeholders:	Students, Faculty Instructors, Academic Administrators
Document Version:	1.0.0 (Release Candidate)
Date:	August 2026

Executive Summary:

This software engineering specification document articulates the complete requirements, system architecture, development methodologies, and structural models for **My Student Academia**. The system modernizes higher-education course enrollment by automating prerequisite validation, dynamic seat allocation, FIFO waitlist queuing, and institution-wide academic administration.

1. Abstract

My Student Academia is a web-based Course Registration and Academic Management System designed to eliminate manual friction in higher-education enrollment workflows. The platform centralizes multi-departmental course offerings, automates prerequisite and credit limit policy checks, manages real-time section capacities, and automates FIFO waitlist queues. By employing a modular 3-tier architecture with role-specific workspaces for Students, Instructors, and Administrators, the platform enhances academic transparency, eliminates concurrency bottlenecks, and provides reliable academic record keeping.

2. Introduction

2.1 Background & Motivation

University course registration is a high-volume, time-sensitive academic operation. In traditional environments, institutions struggle with fragmented course schedules, lack of real-time seat availability tracking, and error-prone manual eligibility checks. When sections reach peak capacity, managing overflow queues without automated waitlists leads to administrative delays and student frustration.

2.2 Problem Statement

Students require immediate feedback on course eligibility, credit caps, and seat availability. Concurrently, academic administrators require a single source of truth to manage courses, balance section loads, and monitor institutional enrollment metrics without manual data reconciliation.

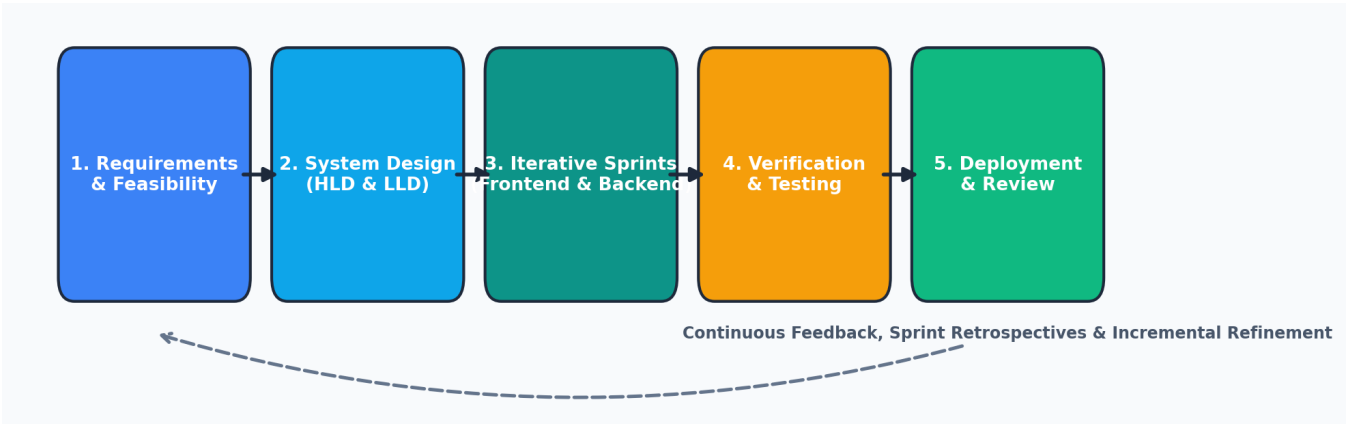
2.3 Project Objectives

- **Automate Registration Workflows:** Single-action course registration with atomic credit limit verification (capped at 24 credits per semester).
- **Enforce Policy Rules:** Automatically block duplicate registrations, enforce prerequisite sequences, and reject credit overloads.
- **Automate Dynamic Waitlisting:** Position students into transparent FIFO waitlists when sections reach full capacity.
- **Role-Based Portals:** Dedicated views for Students (catalog & GPA performance), Instructors (rosters & waitlists), and Administrators (curriculum & analytics).

3. Software Engineering Process Model

3.1 Model Selection: Agile Scrum & Incremental Iterative Model

The project was executed following the **Agile Scrum / Incremental Iterative Process Model**. This model was chosen because academic registration software requires continuous validation of complex business rules (such as credit caps, prerequisite graphs, and capacity ceilings) alongside iterative user interface enhancements.



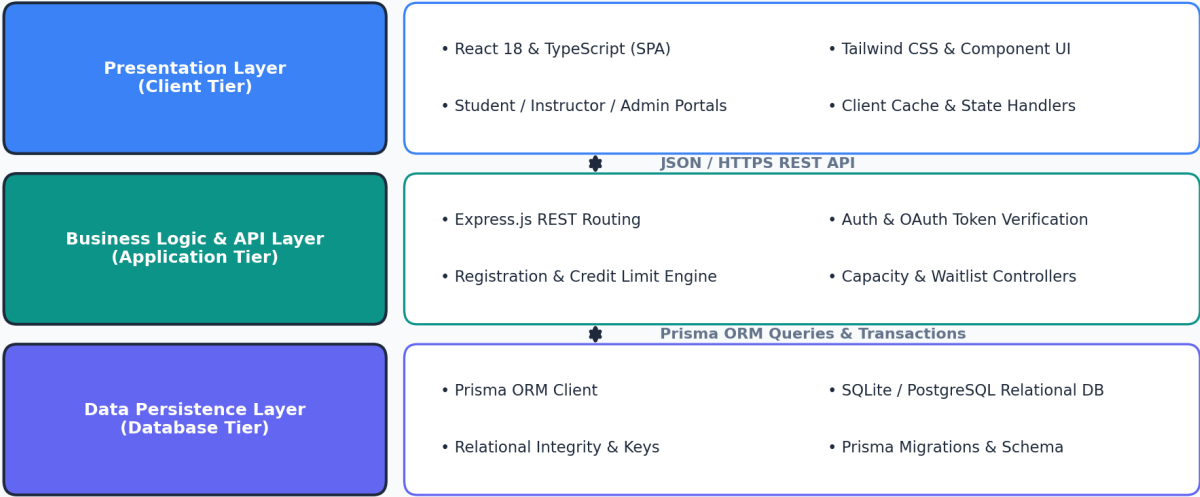
3.2 Lifecycle Phases & Execution Strategy

Phase	Activities & Engineering Artifacts
1. Requirement Engineering	Stakeholder elicitation, user stories, role boundaries (Student, Faculty, Admin), constraint identification (24-credit max).
2. Architectural Design	3-tier client-server decomposition, REST endpoint schema definitions, DFD Level 0/1/2, ER entity modeling.
3. Incremental Sprints	Sprint 1: Auth & Catalog API; Sprint 2: Registration & Waitlist engine; Sprint 3: UI & Dashboards; Sprint 4: Admin analytics.
4. Verification & Testing	Unit testing of registration rules, concurrency validation for seat allocation, integration testing of REST APIs.
5. Deployment & Review	Docker containerization, environment variable configuration, schema migrations, and stakeholder feedback reviews.

4. System Architecture & Software Design Patterns

4.1 3-Tier Layered Architecture

The system adheres to a strict **3-Tier Layered Architecture** ensuring high cohesion, loose coupling, and maintainability across tiers:



4.2 Applied Software Design Patterns

- **Controller-Route Separation Pattern:** Decouples Express HTTP transport routing from business rule execution.
- **Data Mapper / ORM Pattern:** Prisma ORM maps relational database tables directly to strongly typed TypeScript domain models.
- **Component-Based UI Architecture:** Reusable modular React components adhering to single-responsibility principles.
- **Role-Based Access Control (RBAC):** Hierarchical role validation ensuring strict separation of Student, Instructor, and Admin privileges.

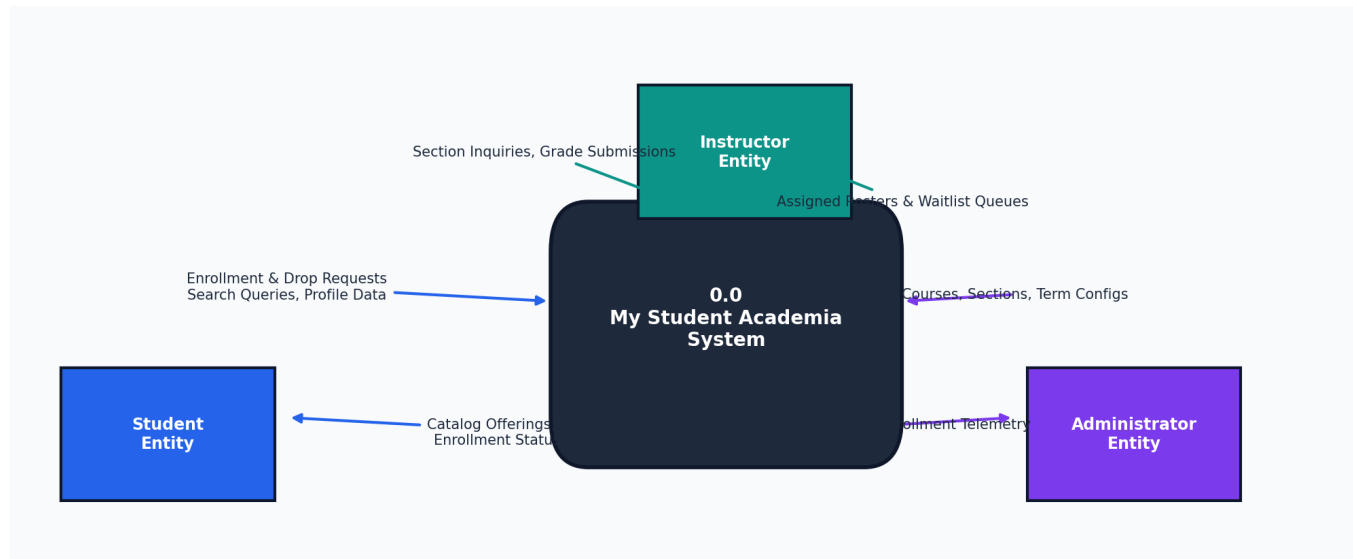
5. Technology Stack & Tools Used

Category	Technology / Tool	Engineering Rationale
Frontend Framework	React 18 + TypeScript + Vite	High performance SPA, static type safety, sub-millisecond HMR build times.
Styling & UX	Tailwind CSS, GSAP, Lucide Icons	Utility-first design, fluid micro-interactions, responsive cross-device layouts.
Backend Runtime	Node.js & Express.js (TypeScript)	Event-driven asynchronous I/O, modular REST architecture, strong typing.
Database & ORM	Prisma ORM with SQLite / PostgreSQL	Declarative schema modeling, automated type generation, transactional safety.
DevOps & Container	Docker & Docker Compose	Consistent runtime isolation across local and production deployment environments.

6. Data Flow Diagrams (DFDs)

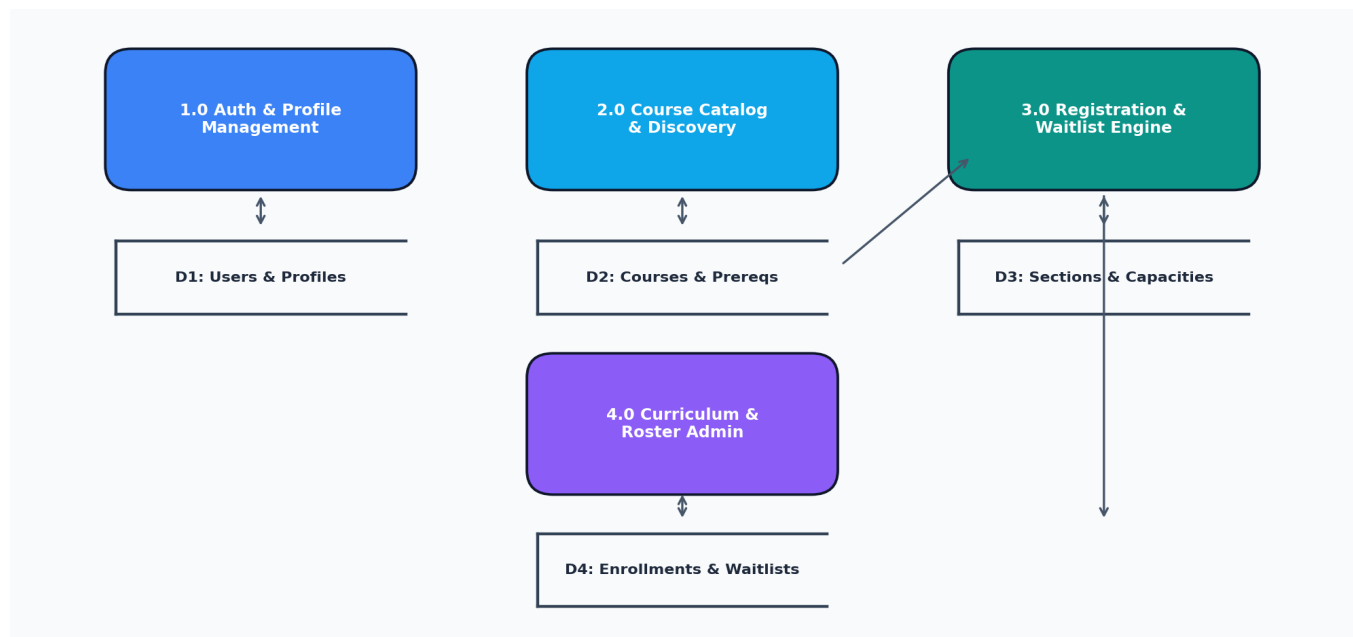
6.1 DFD Level 0: Context Diagram

The Level 0 diagram outlines external boundary interactions with Student, Instructor, and Admin entities.



6.2 DFD Level 1: Subsystem Process Decomposition

Decomposes the system into 4 core functional processes and four underlying data stores.



6.3 DFD Level 2: Course Registration & Waitlist Logic

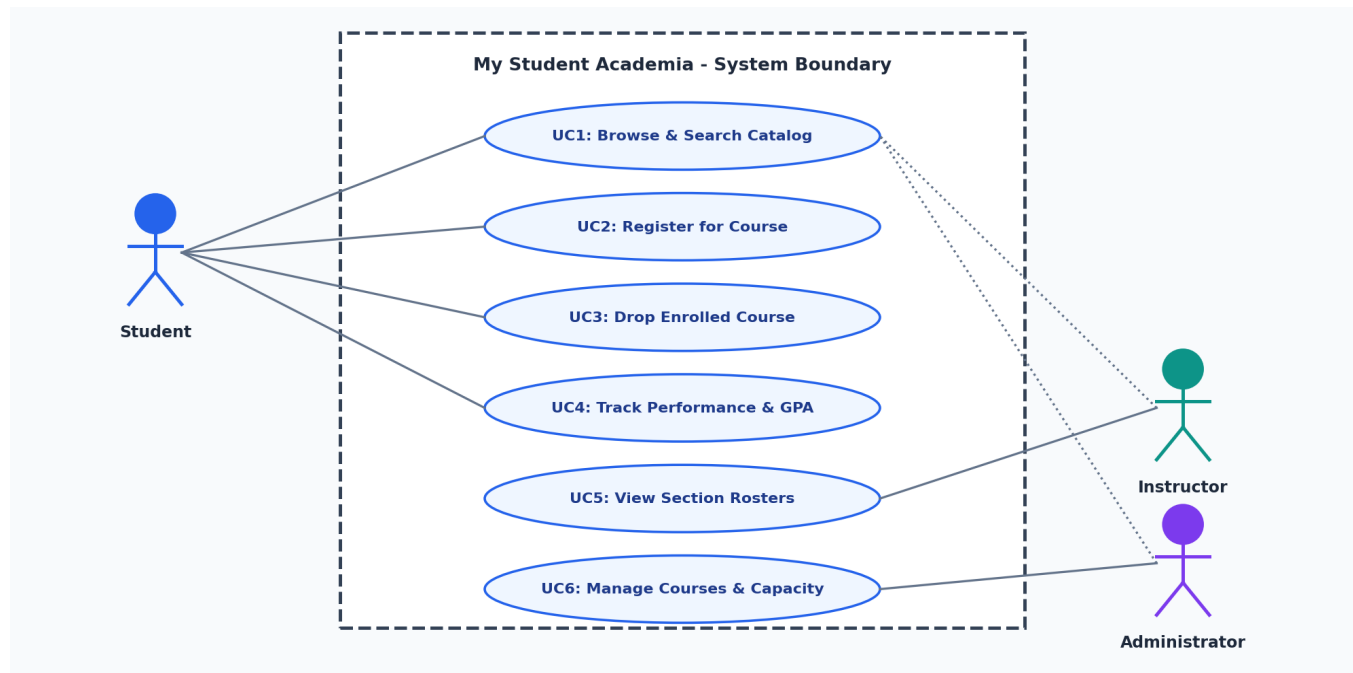
The Level 2 DFD describes the atomic decision tree executed during course registration:

Step / Rule	Validation Logic	Outcome / State Transition
1. Identity Check	Verify authenticated student record.	Proceed or reject with 401 Unauthorized.
2. Duplicate Guard	Check if user already holds status ENROLLED in course.	Reject with 'Already Registered' notification.
3. Credit Limit Check	Check if (Current Credits + Course Credits) <= 24.	Reject if limit exceeded; proceed if within 24 credits.
4. Prerequisite Check	Ensure student completed required prerequisite courses.	Reject if prerequisites missing; proceed if fulfilled.
5. Capacity Check	Check if Section Enrolled Count < Max Capacity.	If Yes: Set status = ENROLLED. If No: Assign next FIFO Waitlist position.

7. Use Case Modeling & Specifications

7.1 Use Case Diagram

Visual mapping of actors (Student, Instructor, Admin) to system use cases:

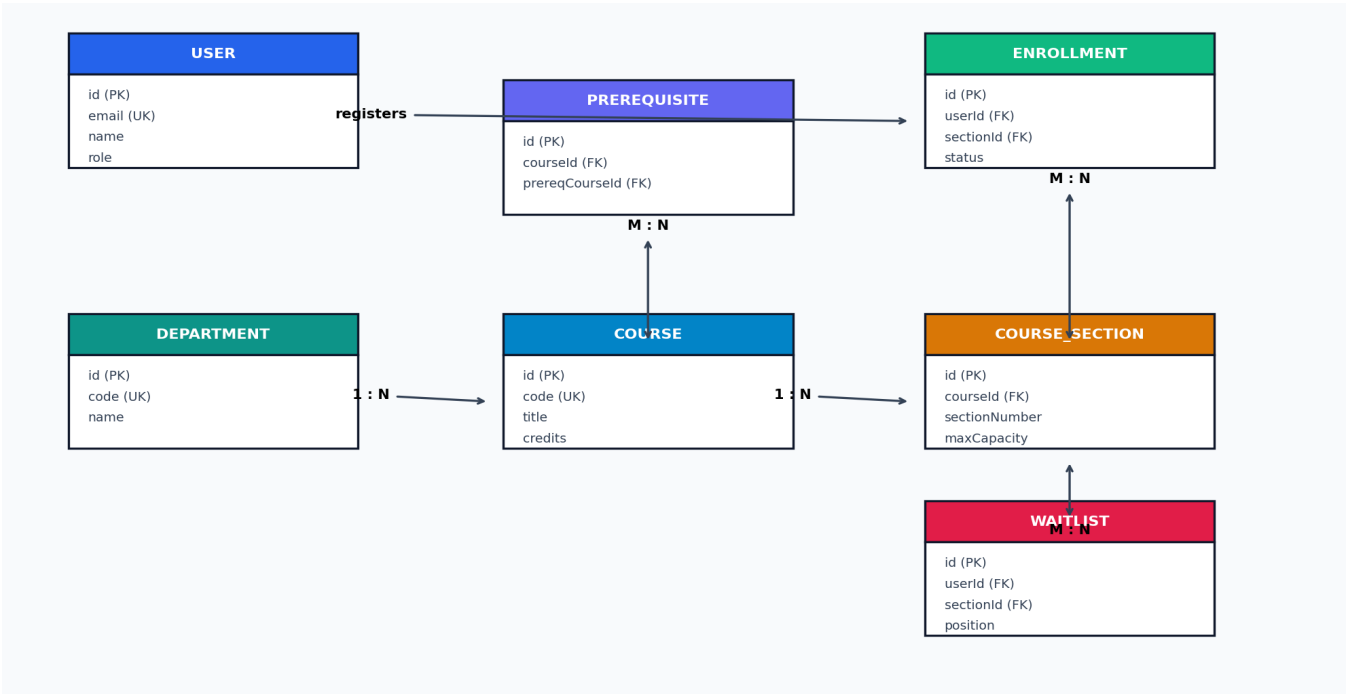


7.2 Detailed Use Case Specifications

Use Case ID:	UC-02: Course Registration
Primary Actor:	Student
Preconditions:	Student is logged in; academic term registration is open.
Main Flow:	1. Student browses catalog and selects desired course section. 2. System validates credit limit (≤ 24) and prerequisite completion. 3. System checks section seat capacity. 4. If seats available, system creates ENROLLED record and updates credits. 5. System displays success confirmation.
Alternate Flow (Waitlist):	If section is full, system places student into FIFO waitlist queue with calculated position.
Postconditions:	Student registration or waitlist state persisted in database.

8. Entity-Relationship (ER) Modeling

8.1 Conceptual & Logical ER Diagram



8.2 Relational Data Dictionary

Entity	Key Attributes & Types	Constraints & Relationships
USER	id (UUID), email (String), name (String), role (Enum), studentId (String), totalCredits (Int)	PK: id; UK: email, studentId. Roles: STUDENT, INSTRUCTOR, ADMIN.
DEPARTMENT	id (UUID), code (String), name (String)	PK: id; UK: code (e.g. CS, AI, MATH). 1:N with Course.
TERM	id (UUID), name (String), isCurrent (Boolean), registrationOpen (Boolean)	PK: id. Represents academic semester.
COURSE	id (UUID), code (String), title (String), credits (Int), departmentId (FK)	PK: id; UK: code; FK: departmentId. 1:N with CourseSection.
PREREQUISITE	id (UUID), courseId (FK), prereqCourseId (FK)	Self-referencing many-to-many relationship on Course.
COURSE_SECTION	id (UUID), courseId (FK), sectionNumber (String), maxCapacity (Int), instructorId (FK)	PK: id; FKs: courseId, termId, instructorId. Holds student capacity.
ENROLLMENT	id (UUID), userId (FK), sectionId (FK), status (String), grade (String)	Unique (userId, sectionId). Status: ENROLLED, DROPPED, COMPLETED.
WAITLIST	id (UUID), userId (FK), sectionId (FK), position (Int), createdAt (DateTime)	Unique (userId, sectionId). Stores FIFO queue position.

9. Non-Functional Requirements & Software Quality Attributes

- **Data Consistency & ACID Compliance:** Transactional integrity guarantees that seat counts and waitlist allocations are updated atomically without race conditions.
- **Role-Based Access Security:** All sensitive administration and grading endpoints require verified identity credentials.
- **Performance & Responsiveness:** Sub-100ms API response times enabled by indexed relational lookups and optimized client-side state handling.
- **Maintainability & Scalability:** Modular layered codebase designed to scale seamlessly to distributed database clusters (PostgreSQL) and enterprise cloud deployments.

10. Conclusion

The **My Student Academia** course registration system provides a robust, scalable, and intuitive platform designed in accordance with rigorous software engineering principles. By uniting modern web technologies, relational integrity, and clear role-based separation of concerns, the system successfully modernizes the academic course lifecycle for students, faculty, and university administrators.